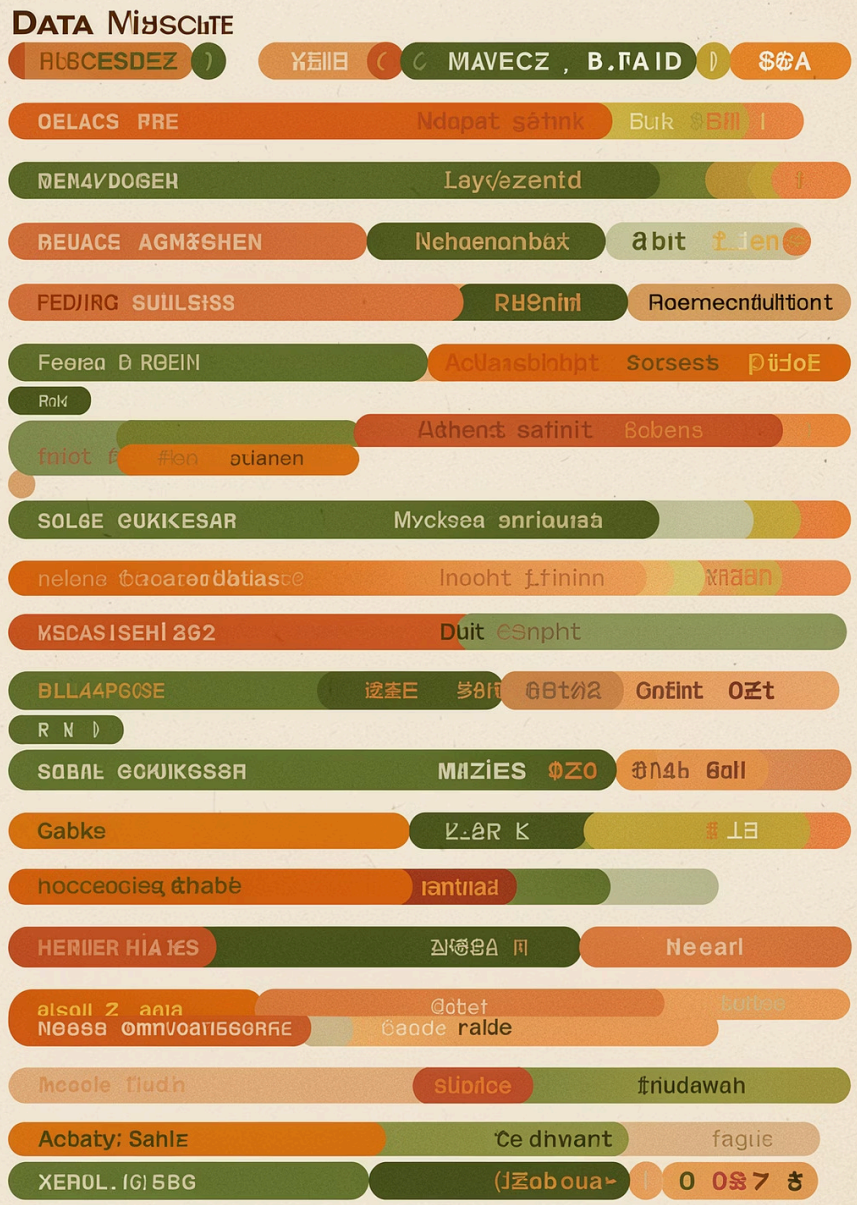


Mastering String Lists

Organizing textual data efficiently in Python and C#

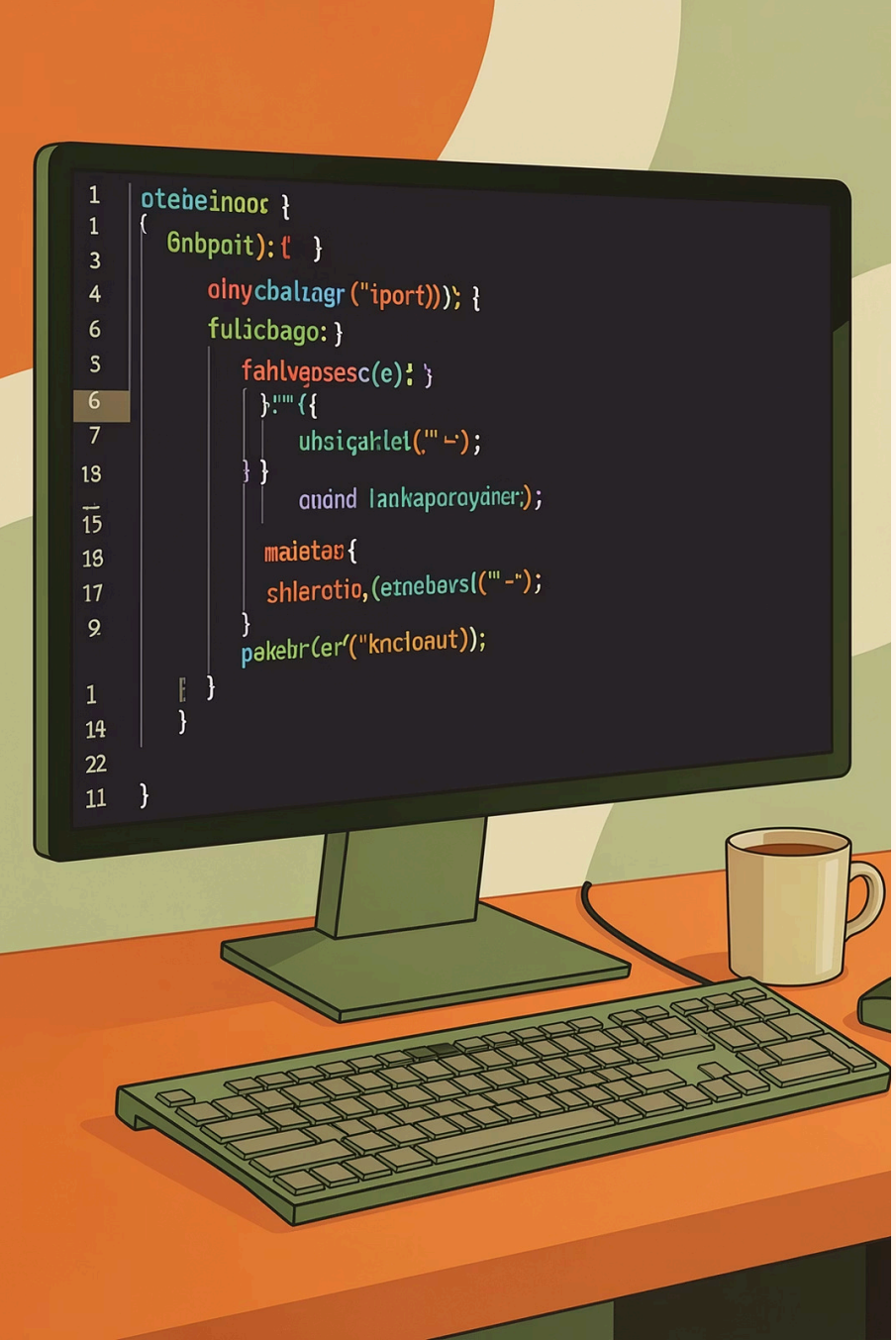
Made with GAMMA

What is a String List?

A **string list** is a collection designed to store multiple strings in a single, ordered structure. Rather than managing dozens of separate variables, a list lets you group related text together and work with it as one unit.

String lists are a foundational tool for text processing, data parsing, and building robust applications — in virtually every modern programming language.

Made with GAMMA



Python: Creating & Manipulating String Lists

Initialization

With a literal: `strings = ["one", "two", "THREE"]`

With `list()` and `append()`: `strings2 = list(); strings2.append("one")`

Combining Lists

Use the `+` operator to merge two lists:

`result = left + right`

Reading from Files

Use `readlines()` paired with `rstrip()` to load lines from a file and strip trailing newline characters automatically.

Made with GAMMA

Python: Iterating & Accessing Elements

Looping Techniques

Made with GAMMA

C#: String Lists in .NET

C# uses the System.Collections.Generic.List<string> class for dynamic string collections — providing strong typing and a rich API.

Adding Elements

```
values.Add("one");
```

Accessing with Index

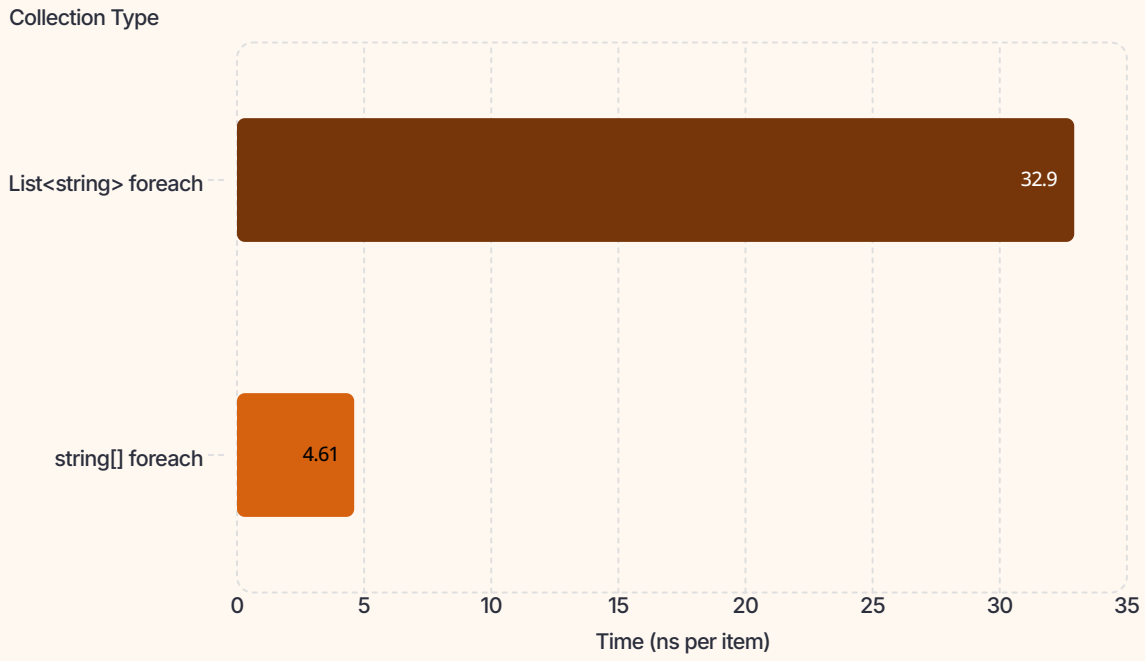
```
string value = values[i];
```

Iterating with foreach

```
foreach (string value in colorsList) { ...
}
```

Made with GAMMA

C#: Lists vs. Arrays — Performance



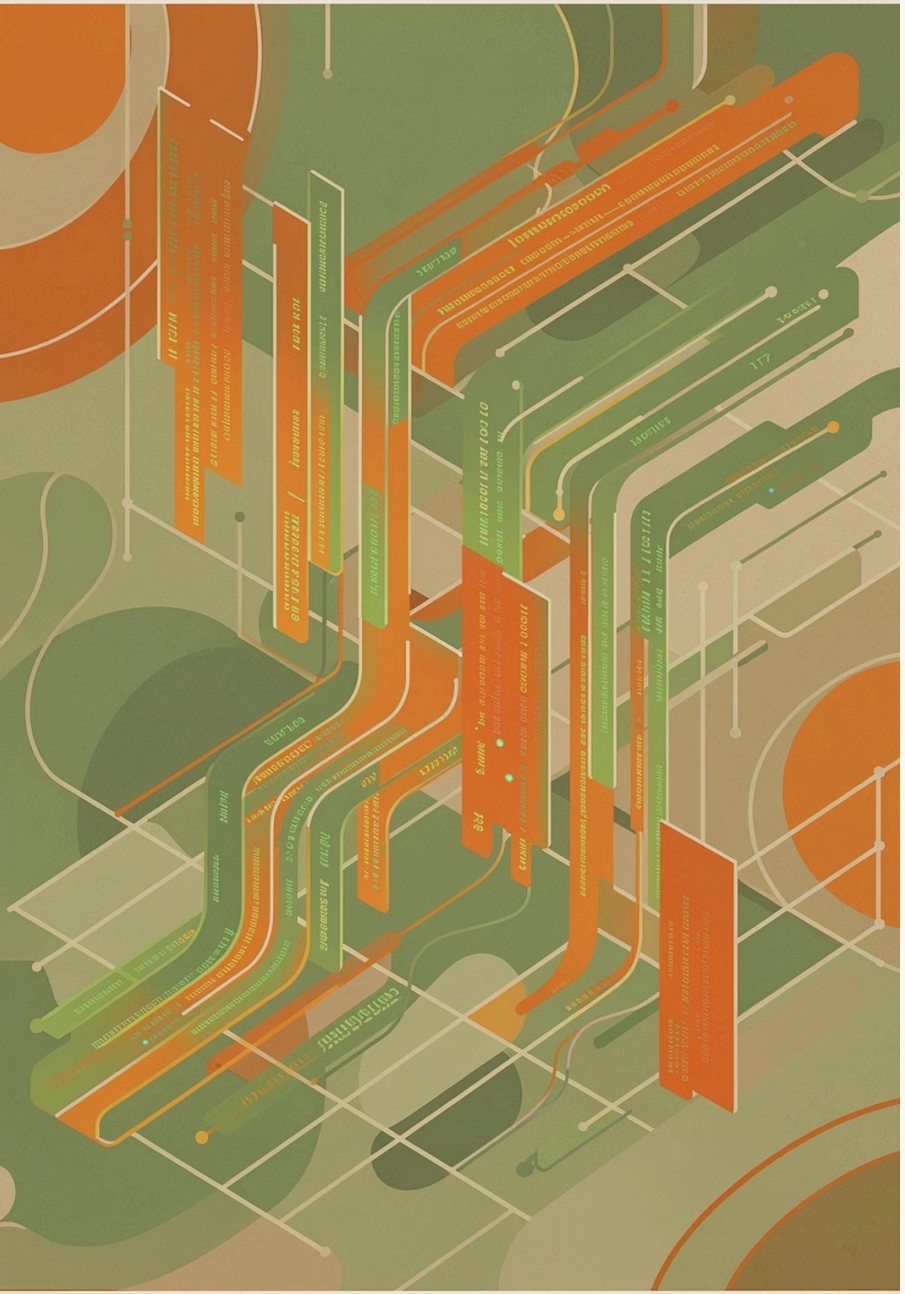
What the Numbers Mean

String Lists (`List<string>`) offer flexibility — adding and removing elements is simple with `Add()` and `Remove()`.

String Arrays are significantly faster for pure iteration — roughly **7× faster** in benchmarks.

⚠ For performance-critical loops over fixed data, prefer `string[]` over `List<string>`.

Made with GAMMA



Organizing Chaos into Order

String lists transform raw, scattered textual data into structured, manageable collections — the backbone of every data-driven application.

Made with GAMMA

Key Takeaways: Python vs. C#

Python

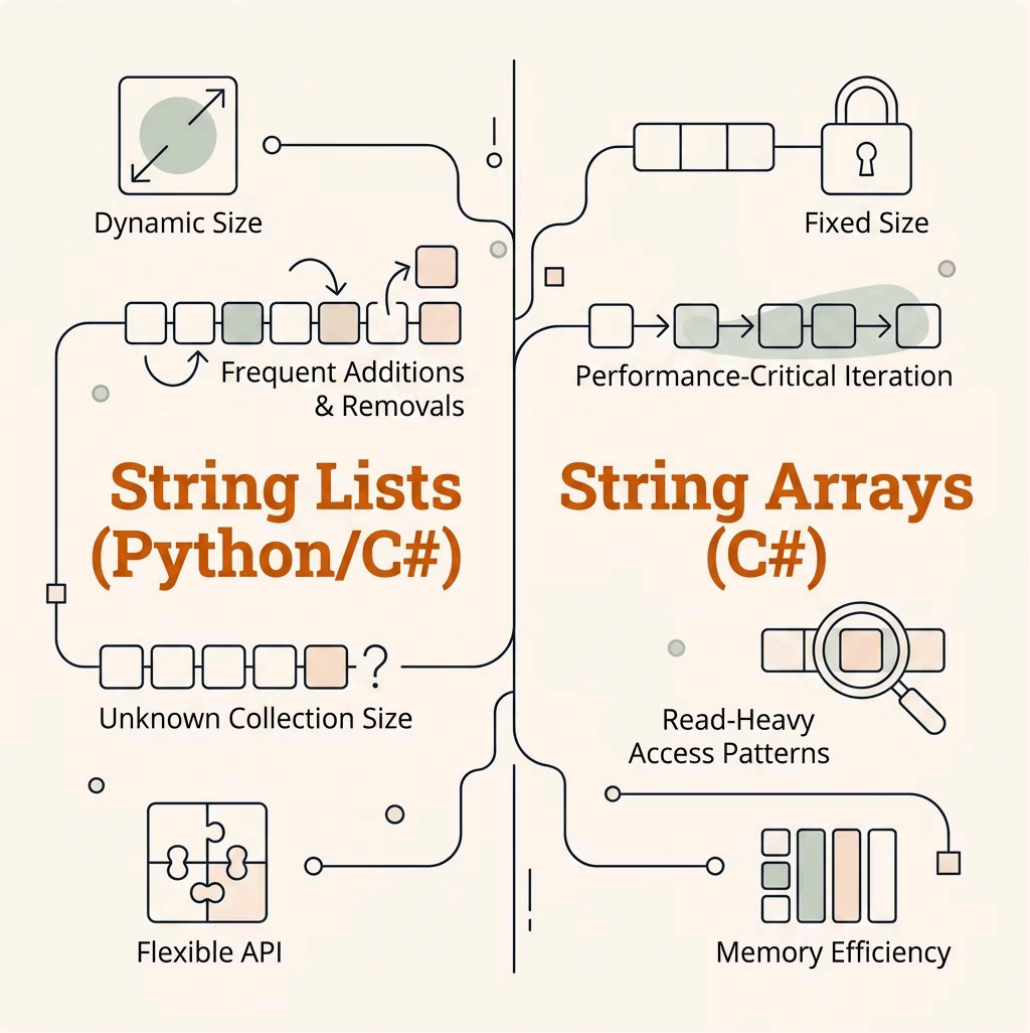
- Dynamic typing — no type declarations needed
- Rich built-in methods for sorting, searching, and modifying
- Concise, readable syntax ideal for rapid development

C#

- Statically typed — compile-time safety with `List<string>`
- Flexible list API with strong performance controls
- Arrays offer a speed advantage for fixed, iterated collections

Made with GAMMA

When to Use Which?



Decision Guide

Choose **String Lists** when your collection grows or shrinks at runtime, or when you need the full suite of manipulation methods.

Choose **String Arrays** in C# when the size is fixed and iteration speed is your top priority — especially in tight loops or benchmarks.

> When in doubt, start with a List — you can always optimize to an array later.

Made with GAMMA

The Power of Organized Text

String lists are among the most fundamental building blocks in programming. From parsing log files to driving machine learning pipelines, mastering lists unlocks efficient, scalable solutions for any text-driven challenge.

Text Processing

Parse, filter, and transform large volumes of textual data with ease.

Data Analysis

Aggregate and organize string data as the first step in any pipeline.

App Development

Power user interfaces, dropdowns, search, and more with dynamic lists.

Made with GAMMA